

Programação Web — JavaScript / PHP / MySQL

Aula 18 — CRUD: Update e Delete (UPDATE + DELETE)

Nome: _____ Data: _____

1. CRUD: Update e Delete

PHP

```

<?php
require_once 'config.php';
$conn = conectar();

// UPDATE: atualizar dados de um aluno
if (isset($_POST['atualizar'])) {
    $id = intval($_POST['id']);
    $nome = htmlspecialchars($_POST['nome']);
    $nota = floatval($_POST['nota']);

    $sql = "UPDATE alunos SET nome='$nome', nota=$nota WHERE id=$id";
    if ($conn->query($sql)) {
        echo 'Atualizado com sucesso!';
    } else {
        echo 'Erro: ' . $conn->error;
    }
}

// DELETE: excluir um aluno pelo ID
if (isset($_GET['excluir'])) {
    $id = intval($_GET['excluir']); // intval() evita SQL Injection básico

    $sql = "DELETE FROM alunos WHERE id=$id";
    if ($conn->query($sql)) {
        echo 'Excluído com sucesso!';
    } else {
        echo 'Erro: ' . $conn->error;
    }
}

// Listar alunos com botões de editar e excluir
$result = $conn->query('SELECT * FROM alunos');
while ($row = $result->fetch_assoc()) {
    echo $row['nome'] . ' | ';
    echo '<a href="editar.php?id=' . $row['id'] . '">Editar</a> | ';
    echo '<a href="?excluir=' . $row['id'] . '" onclick="return confirm(\'"Excluir?"")">';
    echo 'Excluir</a><br>';
}
$conn->close();
?>

```

2. Prevenção básica de SQL Injection

SQL Injection ocorre quando dados do usuário são inseridos diretamente no SQL sem sanitização. A solução correta são prepared statements:

PHP

```
<?php
// FORMA CORRETA: Prepared Statement
$stmt = $conn->prepare(
    'INSERT INTO alunos (nome, email, nota) VALUES (?, ?, ?)'
);
$stmt->bind_param('ssd', $nome, $email, $nota);
// s=string, s=string, d=decimal
$stmt->execute();
$stmt->close();
?>
```

Exercício 1 — Update e Delete

Adicione à listagem de produtos criada na aula anterior botões de 'Editar' e 'Excluir'. O botão excluir deve pedir confirmação antes de deletar:

Resumo da aula

- UPDATE atualiza dados existentes — sempre use WHERE para não alterar todos os registros.
- DELETE remove registros — sempre use WHERE, e confirme com o usuário antes.
- Prepared statements previnem SQL Injection — use sempre em produção.